

Project Title:
Intelligent Job Portal

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
Abhay Tripathi (2520030353)
K.Pranav (2520030477)

Team Number: 9
Section: 6

Under the guidance of

Dr. V. Sireesha
Professor, Department of CSE



Hyderabad-500090, Telangana, India.

ABSTRACT

Modern job portals manage thousands of job postings, requiring highly efficient search and matching mechanisms to connect applicants with relevant opportunities based on skills and location constraints. This project presents Intelligent Job Portal, a lightweight, high-performance recruitment application developed in Java.

To comply with core system constraints, the application operates entirely without external database management systems or built-in collection libraries (`java.util.*`), relying on native Java File Handling (`BufferedReader` and `BufferedWriter`) for flat text file persistence.

Upon startup, system entities—including job listings, user profiles, and application logs—are parsed sequentially from delimited text files into optimized, custom in-memory data structures. The backend architecture integrates key Data Structures and Algorithms (DSA) to deliver fast, algorithmically sound operations:

- **Efficient Indexing & Retrieval:** A custom **Inverted HashMap** maps skills and geographical locations to job listings, achieving $O(1)$ average-time search lookups instead of high-cost $O(N)$ linear file scans.
- **Dynamic Candidate Ranking:** A **Max-Heap (Priority Queue)** evaluates candidate profile vectors against job requirements, dynamically sorting search results to return the top- K best-matched opportunities in $O(K \log N)$ time.
- **Application Pipeline:** A First-In, First-Out (**Queue**) manages sequential job applications before serializing updates back to `applications.txt`.
- **Advanced Text Analytics & Fuzzy Search:** Implements the **Knuth-Morris-Pratt (KMP)** algorithm for linear $O(N + M)$ pattern searching within job descriptions, and **Levenshtein Edit Distance (Dynamic Programming)** to auto-correct user search bar typos.
- **Flow Optimization & Sampling:** Utilizes **Max-Flow Bipartite Matching** (Edmonds-Karp) to compute optimal candidate-to-job allocations under capacity limits, alongside **Reservoir Sampling** to extract unbiased candidate samples directly from raw data streams in a single pass.

Signature of the Guide